Traxer Test Plan

TRX_TSTP

4/21/2020

Author	Changes	Approver	Version Number
Traxer Dev Team	**Initial Draft**	**Traxer Dev Lead**	**Version 1**

Introduction

The purpose of this test plan is to lay out the requirements that must be met while testing the Traxer applications. We have identified the key features that must be tested to ensure user-functionality and system operations. We will be leveraging key standards from the National Institute for Standards and Technology to ensure best practices are employed and the application is secure as well as functional.

To ensure the quality and accuracy of tests we will be leveraging an agile review process wherein features and sections of the application are assigned to teams who will test them, review code, and leverage automatic software. As tests are conducted misconfigurations, issues, or potential improvements will be logged as tickets in our ticketing system assigned to developers of the respective team. From there development follows standard processes for code reviews, dynamic scanning, and approvals.

All changes will be made to the development environment assigned to the development team in question, and at the end of testing code will be merged with the master branch. As code is promoted through the differing environments independent Quality Assurance team members will review and perform additional tests. Prior to any change moving to production a new backup will be created in the passive site running on Microsoft Azure Cloud.

This plan has been reviewed and approved by the Development Lead for the Traxer application after review and approval by the Senior Leadership Team.

Objectives

The objectives are to:

- Identify the activities required to prepare for and conduct testing
- Define the test scope, strategy and methodology to be used for the test
- Identify configuration controls and metrics
- Define metrics and status reporting
- Identify responsibilities for the tasks included in the plan
- Define the test tools and environment needed to conduct the test
- Identify test interactions with other organizations
- Identify test customers and deliverables
- Identify the major testing milestones
- Define the sources of information used to prepare the plan

Scope

The scope of this document is to:

- Explain our approach towards cloud application security and test methodology
- Provide a recommended security test framework to apply to all Traxer applications and dependent infrastructure
- Provide an application tests for the security architecture for applications in PaaS and IaaS with specific recommendations addressing the application components.
- Outline steps for completing effective testing to ensure a secure application.

Functional Test Items

Traxer testing will leverage Functional testing to evaluate the compliance of the system or component with specified functional requirements. Functional testing usually describes what the system does in several areas of interconnected components.

The Traxer testing team will have functional tests and unit tests, functional tests will include the following:

- Access Control Testing
- Benchmark Testing
- Build Verification Testing
- Business Cycle Testing
- Compliance Testing
- Component Integration
- Configuration Testing

- Contention Testing
- Data and Database Integrity Testing
- Document Management Testing
- Error Analysis Testing
- Exploratory Testing
- Failover Testing
- Installation Testing
- Integration testing
- Parallel Testing
- Performance Monitoring Testing
- Privacy Testing
- Product Component Testing
- Recovery Testing
- Regression Testing
- Risk Based Testing
- Security Testing

Functional Test Items Definition

The table below represents the functional tests and its detailed definition.

Functional Tests	Definition
Access Control Testing	A type of testing that attests that the target-of-test data (or systems) are accessible only to those actors for which they are intended, as defined by use cases. Access Control Testing verifies that access to the system is controlled and that unwanted or unauthorized access is prohibited. This test is implemented and executed on various Traxer endpoints.
Benchmark Testing:	A type of performance testing that compares the performance of new or unknown functionality to a known reference standard (e.g., existing software or measurements). For example, benchmark testing may compare the performance of current systems with the performance baseline.
Build Verification Testing (Prerequisite: Smoke Test)	A type of testing performed for each new build, comparing the baseline with the actual object properties in the current build. The output from this test indicates what object properties have changed or don't meet the requirements. Together with the Smoke test, the Build Verification test may be utilized by projects to determine if additional functional testing is appropriate for a given build or if a build is ready for production.

Functional Tests	Definition
Business Cycle Testing	A type of testing that focuses upon activities and transactions performed end-to-end over time. This test type executes the functionality associated with a period of time (e.g., one-week, month, or year). These tests include all daily, weekly, and monthly cycles, and events that are date-sensitive (e.g., end of the month management reports, monthly reports, quarterly reports, and year-end reports).
Compliance Testing	A type of testing that verifies that a collection of software and hardware fulfills given specifications. It does not exclude any other tests that may also come up.
Component Integration Testing	Testing performed to expose defects in the interfaces and interaction between integrated components as well as verifying installation instructions. Extremely important test to test exposure of endpoints.
Configuration Testing	A type of testing concerned with checking the programs compatibility with as many possible configurations of hardware and system software. In most production environments, the particular hardware specifications for the client workstations, network connections, and database servers vary. Client workstations may have different software loaded, for example, applications, drivers, and so on hand, at any one time; many different combinations may be active using different resources. The goal of the configuration test is finding a hardware combination that should be, but is not, compatible with the program.
Contention Testing	A type of performance testing that executes tests that causes the application to fail with regard to actual or simulated concurrency. Contention testing identifies failures associated with locking, deadlock, livelock, starvation, race conditions, priority inversion, data loss, loss of memory, and lack of thread safety in shared software components or data.
Data and Database Integrity Testing	A type of testing that verifies that data is being stored by the system in a manner where the data is not compromised by the initial storage, updating, restoration, or retrieval processing. This type of testing is intended to uncover design flaws that may result in data corruption, unauthorized data access, lack of data integrity across multiple tables, and lack of adequate transaction performance. The databases, data files, and the database or data file processes should be tested as a subsystem within the application.

Functional Tests	Definition
Documentation Testing	Documentation testing is a type of testing that should validate the information contained within the software documentation set for the following qualities: compliance to accepted standards and conventions, accuracy, completeness, and usability. The documentation testing should verify that all of the required information is provided in order for the appropriate user to be able to properly install, implement, operate, and maintain the software application.
Error Analysis Testing	This type of testing verifies that the application checks for input, detects invalid data, and prevents invalid data from being entered into the application. This type of testing also includes the verification of error logs and error messages that are displayed to the user.
Exploratory Testing	A technique for testing computer software that requires minimal planning and tolerates limited documentation for the target-of-test in advance of test execution, relying on the skill and knowledge of the tester and feedback from test results to guide the ongoing test effort. Exploratory testing is often conducted in short sessions in which feedback gained from one session is used to dynamically plan subsequent sessions.
Failover Testing	A type of testing test that ensures an alternate or backup system properly "takes over" (i.e., a backup system functions when the primary system fails). Failover Testing also tests that a system continually runs when the failover occurs, and that the failover happens without any loss of data or transactions. Failover Testing should be combined with Recovery Testing.
Installation Testing	A type of testing that verifies that the application or system installs as intended on different hardware and software configurations, and under different conditions (e.g., a new installation, an upgrade, and a complete or custom installation). Installation testing may also measure the ease with which an application or system can be successfully installed, typically measured in terms of the average amount of person-hours required for a trained operator or hardware engineer to perform the installation. Part of this installation test is to perform an uninstall. As a result of this uninstall, the system, application and database should return to the state prior to the install.
Integration Testing	An incremental series of tests of combinations or sub-assemblies of selected components in an overall system. Integration testing is incremental in a successively larger and more complex combinations of components tested in sequence, proceeding from the unit level (0% integration) to eventually the full system test (100% integration).

Functional Tests	Definition
Load Testing	A performance test that subjects the system to varying workloads in order to measure and evaluate the performance behaviors and abilities of the system to continue to function properly under these different workloads. Load testing determines and ensures that the system functions properly beyond the expected maximum workload. Additionally, load testing evaluates the performance characteristics (e.g., response times, transaction rates, and other time-sensitive issues).
Migration Testing	A type of testing that follows standards operating procedures that allows to migrate and scale functions. The following are examples of the types of tests that can be performed as part of migration testing: Data conversion has been completed Data tables are successfully created Parallel test for confirmation of data integrity Review output report, before and after migration, to confirm data integrity Run equivalent process, before and after migration
Multi-Divisional Testing	A type of testing that ensures that all applications will operate in a multi-division or multi-site environment recognizing that an enterprise perspective while fully supporting local health care delivery.
Parallel Testing	The same internal processes are run on the existing system and the new system. The existing system is considered the “gold standard”, unless proven otherwise. The feedback (expected results, defined time limits, data extracts, etc) from processes from the new system are compared to the existing system. Parallel testing is performed before the new system is put into a production environment.
Performance Monitoring Testing	Performance profiling assesses how a system is spending its time and consuming resources. This type of performance testing optimizes the performance of a system by measuring how much time and resources the system is spending in each function. These tests identify performance limitations in the code and specify which sections of the code would benefit most from optimization work. The goal of performance profiling is to optimize the feature and application performance.
Performance Testing	Performance Testing assesses how a system is spending its time and consuming resources. Performance testing optimizes a system by measuring how much time and resources the system is spending in each function. These tests identify performance limitations in the code and specify which sections of the code would benefit most from optimization work. Performance testing may be further refined by the use of specific types of performance tests, such as, benchmark test, load test, stress test, performance monitoring test, and contention test.

Functional Tests	Definition
Privacy Testing	A type of testing that ensures that data are adequately protected, and systems and applications comply with relevant privacy laws.
Product Component Testing	Product Component Testing (aka Unit Testing) is the internal technical and functional testing of a module/component of code. Product Component Testing verifies that the requirements defined in the detail design specification have been successfully applied to the module/component under test.
Recovery Testing	A type of testing that causes an application or system to fail in a controlled environment. Recovery processes are invoked while an application or system is monitored. Recovery testing verifies that application or system, and data recovery is achieved. Recovery Testing should be combined with Failover Testing.
Regression Test	A type of testing that validates existing functionality still performs as expected when new functionality is introduced into the system under test.
Risk Based Testing	A type of testing based on a defined list of project risks. It is designed to explore and/or uncover potential system failures by using the list of risks to select and prioritize testing.
Section 508 Compliance Testing	A type of test that ensures that persons with disabilities have access to and are able to interact with graphical user interfaces and verifies that the application or system meets the specified Section 508 Compliance standards.
Security Testing	A type of test that validates the security requirements and to ensure readiness for the independent testing performed by the Security Assessment Team as required by the Assessment and Authorization Process.
Smoke Test	A type of testing that ensures that an application or system is stable enough to enter testing in the currently active test phase. It is usually a subset of the overall set of tests, preferably automated, that touches parts of the system in at least a cursory way.
Stress Testing	A performance test implemented and executed to understand how a system fails due to conditions at the boundary, or outside of, the expected tolerances. This failure typically involves low resources or competition for resources. Low resource conditions reveal how the target-of-test fails that is not apparent under normal conditions. Other defects might result from competition for shared resources (e.g., database locks or network bandwidth), although some of these tests are usually addressed under functional and load testing. Stress Testing verifies the acceptability of the systems performance behavior when abnormal or extreme conditions are encountered (e.g., diminished resources or extremely high number of users).

Functional Tests	Definition
System Testing	System testing is the testing of all parts of an integrated system, including interfaces to external systems. Both functional and structural types of testing are performed to verify that the system performance, operation and functionality are sound. End to end testing with all interfacing systems is the ultimate version.
Usability Testing	Usability testing identifies problems in the ease-of-use and ease-of-learning of a product. Usability tests may focus upon, and are not limited to: human factors, aesthetics, consistency in the user interface, online and context-sensitive help, wizards and agents, user documentation.
User Functionality Test	User Functionality Test (UAT) is a type of Acceptance Test that involves end-users testing the functionality of the application using test data in a controlled test environment.
User Interface Testing	User-interface (UI) testing exercises the user interfaces to ensure that the interfaces follow accepted standards and meet requirements. User-interface testing is often referred to as GUI testing. UI testing provides tools and services for driving the user interface of an application from a test.

For the functional tests, the Traxer testing team will be leveraging open-source testing tools.

Features to Be Tested

The features being tested are as follows:

- User Account Creation in Customer Tenant (High)
- User Account Login (High)
- Creating Vendor Folders (High)
- Generating Vendor Invitation Emails (Medium)
- Standard Selection (High)
- Questionnaire Creation (Medium)
- Questionnaire Completion (Medium)
- Document Uploading (High)
- Document Deletion (High)
- Document Request (Medium)
- Document Encryption (High)
- Machine Learning Processes (High)

Features Not to Be Tested

Features that will not be tested are:

- General UI (These are standard per system and should not require significant support)
- Customer Access to Tenant (This is set and tested by the support team at time of creation)
- Customer-created Customer Accounts (This access should be set and managed by the customer)

Unit Tests

Unit testing is a software testing method where individual units or components of the software are tested with associated control data using operating procedures. Its focus is to validate whether each group or component of the software performs as designed. It usually consists of one or a few inputs and a single output. Unit testing is an essential part of the development process; it fixes bugs early in the development cycle and saves costs. It also encourages developers to write reusable code. There are two types of unit testing manual and automated. In manual testing, a step-by-step instructional document is created to test the code.

Test ID	**Test Scenario**	**Test Steps**	**Test Data**	**Expected Results**
1	Validate MFA code generation.	Enter AD account – username and password.	Valid test user / test password	Successfully logged into the Tracker Analytics Web and Display dashboard.
		The system sends MFA code to the user's cell phone.	Invalid test user / test password	User redirected to login page with a message "User/Password Invalid".
		The user enters the MFA code.		MFA code successfully sent to the user.
				Unable to send MFA code to the user, display "Issue sending MFA code please contact support".
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
2	Retrieve user privileges based on the role.	The system retrieves access privileges from Active Directory based on the user role.		Successfully retrieved user access privileges and the list of available services displayed.
		Available services are listed on the		No user access privileges found, and a

		dashboard.		message “Contact system administrator for access rights” is displayed.
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
3	Azure Service Bus (ASB) publisher for supplier detail topic.	ASB receives the publisher information for supplier and stores in the queue.	Valid supplier ID	Successfully stores the supplier information received by ASB and stores it in the queue.
			Invalid supplier ID	Unable to add the supplier request into the queue because of an internal issue.
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
4	Supplier Service process subscribes to supplier details topic in ASB.	Azure Function (Supplier Service) subscribes to suppler detail topic in the ASB.	Valid supplier ID	Successfully subscription to the supplier detail topic in the ASB.
		Retrieve the supplier request from the ASB queue.	Invalid supplier ID	Display subscription error message “Unable to subscribe to supplier detail topic” contact support.
				Successfully retrieves the supplier request.
				Unable to retrieve the supplier request, display “ASB subscription error”.
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
5	Retrieve connection string from Azure Key Vault.	Validate the storage account associated with supplier service Azure function.	Valid storage account	Successfully retrieves the connection string using Azure storage account.
		Using storage	Invalid storage	Unsuccessful in

		account, retrieve the database connection string from AKV to connect to Azure SQL for Traxer.	account	retrieving the connection string using Azure storage account. Display "invalid Azure storage account".
				Unsuccessful in retrieving the connection string using Azure storage account. Display "Connection string not found".
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
7	Connect to Azure SQL DB Traxer from the supplier service component to retrieve supplier details.	Using connection string retrieved from AKV, connect to Cosmos DB Traxer.	Valid supplier ID	Successfully retrieves the connection string using Azure storage account.
		Query the Azure SQL DB Traxer for supplier details based on the supplier id presented by the user.	Invalid supplier ID	Successfully connects to Cosmos DB Traxer
				Successfully retrieves order details.
				Unable to connect to Cosmos DB Traxer, display "connection string invalid".
				Unable to retrieve supplier details, display "Invalid supplier ID".
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
8	Connect to Google Maps API to display real-time supplier information on the map inside	Using storage account retrieve API key from the AKV to connect to Google Maps API service.	Valid storage account	Successfully retrieves the API key using Azure storage account.

	the dashboard and identify from where the vendor submitted the documentation	Display supplier information retrieved from Cosmos DB TRAXER on to Google Maps using Google Maps API inside the dashboard.	Invalid storage account	Unsuccessful in retrieving the API key using Azure storage account. Display "invalid Azure storage account".
				Unsuccessful in retrieving the API key using Azure storage account. Display "API key not found".
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
9	Retrieve cyber threat intelligence feeds using APIs.	Risk analysis services Azure function connects to AKV to retrieve API key to connect to cyber threat intelligence feeds.	Valid storage account	Successfully retrieves the API key using Azure storage account.
		Retrieve the feed data and pass it to Azure stream analytics.	Invalid storage account	Unsuccessful in retrieving the API key suing Azure storage account, display "Invalid Azure storage account".
				Successfully passed the data to Azure stream analytics.
				Unable to pass the data to Azure stream analytics, display "Internal issue, contact support".
Test ID	Test Scenario	Test Steps	Test Data	Expected Results
10	Process cyber threat intelligence feeds and save	Azure Stream Analytics processes the cyber threat	Valid cyber threat intelligence data.	Successfully process the cyber threat intelligence data.

	it to Cosmos DB TAD.	intelligence data.		
		Using connection string retrieved from AKV, connect to Cosmos DB Traxer.	Invalid cyber threat intelligence data.	Unable to process the cyber threat intelligence data, display “Invalid data”.
		Write the process data to Cosmos DB Traxer.	Valid connection string.	Successfully retrieves the connection string using Azure storage account.
			Invalid connection string.	Unable to connect to Cosmos DB Traxer, display “connection string invalid”.
				Successfully connects to Cosmos DB Traxer.
				Successfully writes the processed data to Cosmos DB Traxer.
				Unable to write process data to Cosmos DB Traxer, display “Contact support”.

Suspension Criteria and Resumption Requirements

Regression testing is the rerun of functional and non-functional tests to verify that previously developed, and tested software continues to perform after a change. Typically, software changes under which regression testing performed include:

- Change in code due to requirements and configuration changes
- Software enhancements to develop new code or features
- Code modification to fix defects or bugs
- Code modification to fix performance issues

Regression testing techniques generally involve prioritizing high impact test cases, selecting test cases that are impacted by the changes in the code, and lastly, considers the Re-test (all) condition. Automated Testing is the most preferred method of regression testing in software engineering, which depends on the number of test use cases for functional and regression testing; this is the most accurate

method of regression testing and the quickest compared to manual regression testing. (Guru99, n.d.).

Some examples of regression testing tools include:

- Selenium - is a free open source tool that can be used for browser-based regression testing. This tool is compatible and can support many programming languages and operating systems platforms throughout the regression tests in software engineering to automate web application testing. (Selenium, 2020)

- Ranorex Studio - at an affordable price, this comprehensive regression test tool can automate testing for web, desktop, and mobile applications, which includes full IDE tools for codeless automation.

This regression test tool encompasses combability and support in many programming languages and operating systems platforms. (Ranorex, 2020)

Manual Testing is the least preferred method in software engineering regression testing if the software endures many changes throughout the software development life cycle. This method causes regression tests to become costly in time and resources, as test cases can take longer to execute. (Guru99, n.d.).

Test Deliverables

The deliverables of the overall testing plan will be:

- Test plan
- Test case specifications
- Test procedure specifications
- Test logs
- Test Incident Reports
- Test Summary reports

Environmental Needs

All tests will be done throughout the development phase. Final tests will be executed in the Quality Assurance environment prior to code being promoted to production.

Tools

- Veracode
- Kali Linux
- Microsoft’s Security Risk Detection (MSRD)
- Open source for Functional Tests
- Scripts to perform the Unit tests

Data

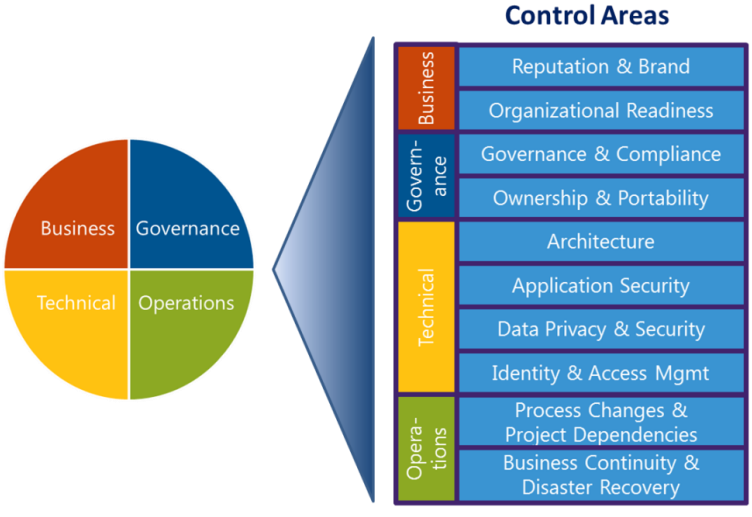
To test our application, we will be leveraging a series internal documents and manually created test data where necessary. We will create several fake customer tenants, sub-customer accounts, and vendors all with the relevant data points included. No data used will be from real customers, and preferably will be entirely false in nature.

Security

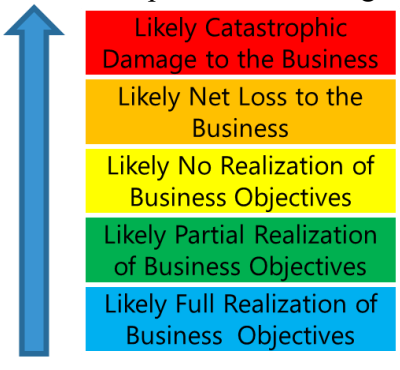
Security is the highest priority next to functionality for this program. Our platform will host sensitive and high-value documentation and security assessments and must be sure. Accordingly, we will be leveraging the Kali Linux suite and its accompanying tools to stress-test our systems. We will execute standard vulnerability scans against the application and attempt more direct penetration attempts where appropriate.
For each component of the system, we will research potential vulnerabilities using public vulnerability databases, such as the Open Source Vulnerability Database (OSVDB), MITRE's Common Vulnerabilities and Exposures (CVE) database, or the Common Weakness Enumeration (CWE) list. Once research is completed, we will test each service for potential vulnerabilities using automated and manual techniques.
All tools and exploits are tested in the testing environment prior to use to ensure the safety and integrity. Each potential issue's security implications are considered in its environment. In this phase, vulnerabilities are correlated with data obtained from the Information Gathering phase, client's inventory, system knowledge, and available network configuration and topology data to reduce false positives and detect false negatives. Vulnerabilities will be thoroughly tested where possible to ensure a high confidence of accurate results.

Microsoft's Security Risk Detection (MSRD)

The methodology that will be used throughout all phases of the testing phase and aa proven Agile Method for Analyzing Risk in Cloud Computing is used to pinpoint which business areas face with the most serious risks and it defines how to mitigate those risks to successfully test everything on the cloud. The risk analysis is in no way intended to replace formal enterprise risk assessments or serve as the only input mechanism for cloud investment decisions. Instead, it is intended to serve as a substantial component of the cloud computing business case.
The development of these business cases is dependent to three main activities: portfolio analysis and application segmentation, investment prioritization (including return on investment calculations), and detailed feasibility and risk analysis.



The Risk-Based Model for Cloud Computing uses a structured approach to risk management, as recommended by Microsoft best practices. This structure is based on a set of control areas, outlined in figure above, which represent different parts of the enterprise and serve as the foundation for the risk analysis. Each initiative is assessed to determine the level of exposure across the control areas.
At the enterprise level, the organization's risk tolerance is assessed for each of the control areas.



Discipline as a function of the probability and impact of the risks. Risk Probability is a measure of the likelihood that the risks associated with the assessed control area will occur. Risk Impact is an estimate of the severity of adverse effects, the magnitude of a loss, or the potential opportunity cost should a risk be realized within a project.
In the Risk-Based Model for Cloud Computing, Risk Exposure (probability * impact) is represented in a five-point scale based on the level of impact a risk has on the business objectives, as shown in figure below.
This scale ranges from no exposure or Likely Full Realization of Business Objectives to very high exposure or Likely Catastrophic Damage to the Business. The levels are color-coded from blue to red which represents cool to hot in relation to the "temperature" of the risk.
A key factor in the overall risk assessment process is determining an enterprise's tolerance for risk. Any organization that is considering moving its workloads to the cloud will need to determine its risk tolerance in each control area. This tolerance is determined at an enterprise level, not at the initiative level.
When analyzing the risks of an individual workload, the risk exposure levels can be compared against the enterprise tolerances to determine whether the risks can be accepted or if mitigation is required.

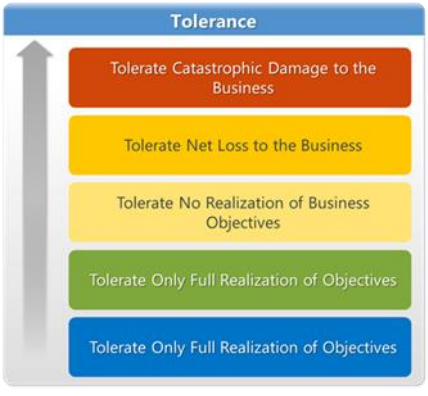


Figure ... Risk tolerance scale

The expected result of all tests is expected to fit in the table like below using major and minor categories. The results below are merely figurative and do not represent the results of the tests from Traxer.

Cloud Risk Control Area		Risk Exposure
Business	Reputation & Brand	*Very High*
	Organizational Readiness	*Low*
Governance	Governance & Compliance	*Moderate*
	Ownership & Portability	*Low*
Technical	Architecture	*Moderate*
	Application Security	*Moderate*
	Data Privacy & Security	*Moderate*
	Identity and Access Management	*Moderate*
Operations	Process Changes & Dependencies	*Moderate*
	Business Continuity & Recovery	*High*

Responsibilities

Role	Description
Development Team	Persons that build or construct the product/product component.
Development Manager	Person responsible for assisting with the creation and implementation of the Master Test Plan.
Program Manager	Person that has overall responsibility for the successful planning and execution of a project; person responsible for creating the Master Test Plan in collaboration with the Development Manager.
Stakeholders	Persons that hold a stake in a situation in which they may affect or be affected by the outcome.
Test Analyst	Person responsible for ensuring full execution of the

Role	Description
	test process to include the verification of technical requirements and the validation of business requirements.
Test Lead	An experienced Test Analyst or member of the Test Team that leads and coordinates activities related to all aspects of testing based on an approved Master Test Plan and schedule.
Test Team/Testers	Persons that execute tests and ensure the test environment will adequately support planned test activities.
Test Environment Team	Persons that establish, maintain, and control test environments.
Quality Assurance Team	Team responsible for the overall quality assurance

Staffing and Training Needs

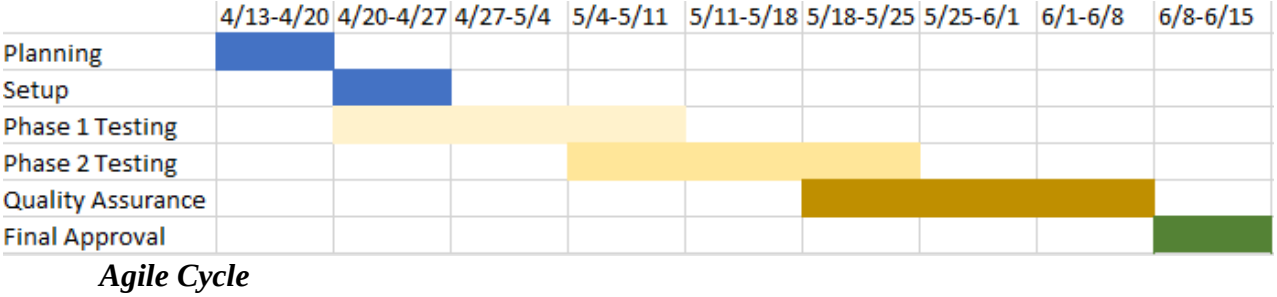
Staff must have appropriate training in the following areas:

- Training on the product.
- Secure Development training
- Training on the dynamic code review program
- Training on Data Privacy requirements

Schedule

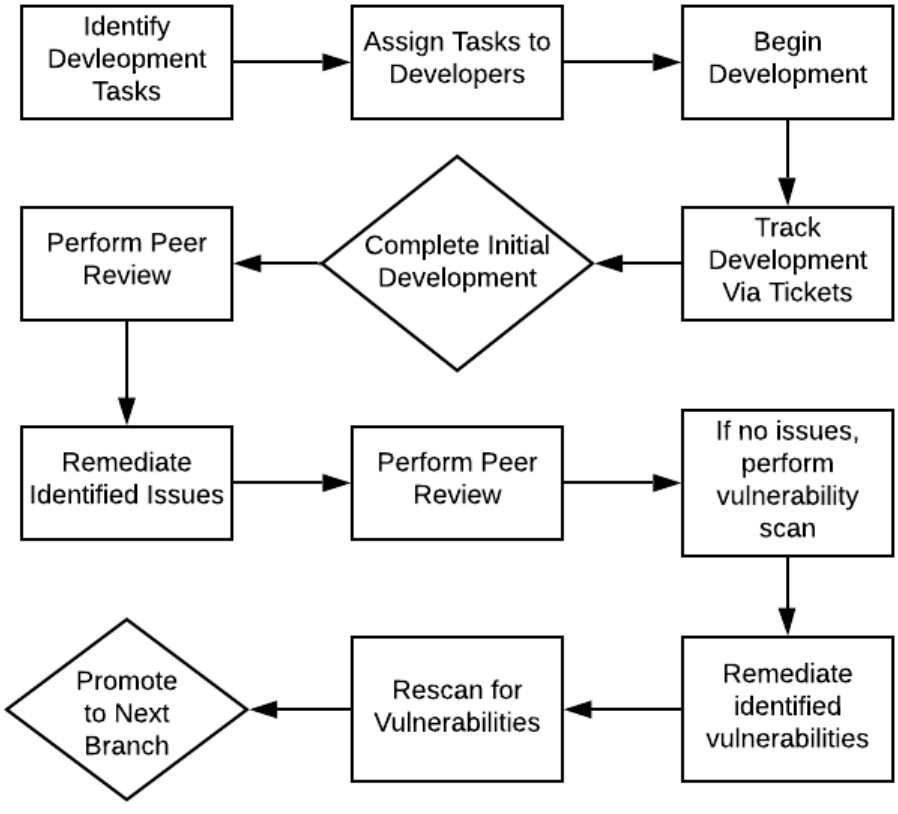
This section describes the overall testing timeline as well as a breakdown of the testing workflow schedule.

Timeline



Agile Cycle

As we progress through the various functional and unit testing our teams will be following the below general workflow:



If overrun occurs or the initial timeline cannot be met the schedule will be re-evaluated. The schedule has some flexibility and can extend to provide more time where necessary. Many of the items identified for testing should not require significant time for testing which free up time for more challenging areas.

Risks and Contingencies

Potential risks and contingencies are outlined in the following sections.

Risks

A list of potential risks that can occur are as follows:

- Damage to application infrastructure
- Loss or corruption of code
- Loss of system functionality
- Loss or corruption of data
- Testing and development create new vulnerabilities

Contingencies

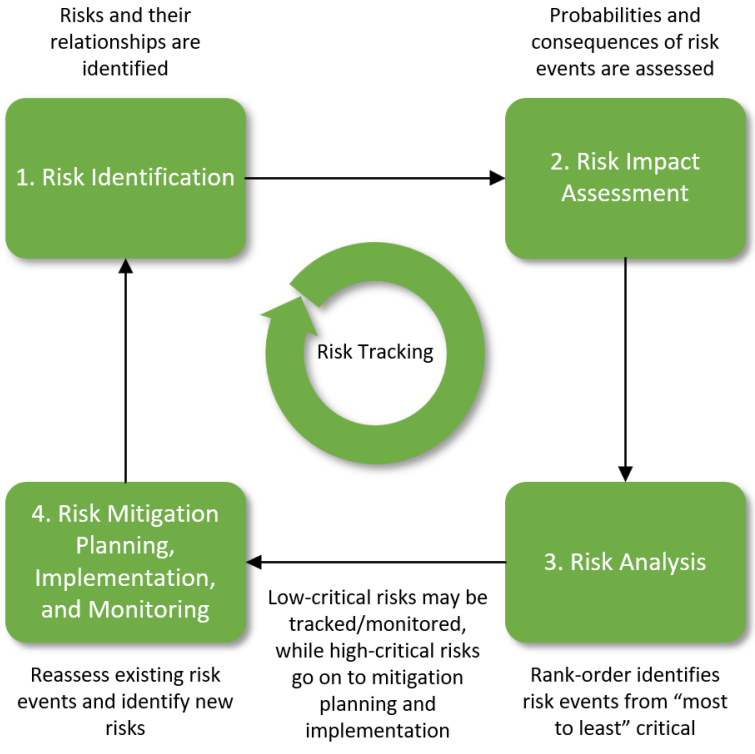
Before any testing begins developers must assure that:

1. The environment is hosted in a sandboxed tenant
2. The environment is created from a recent backup
3. The test does not include or require real data, or any data subject to regulations (e.g. healthcare data or sensitive financial details)
4. Testing accounts does not leverage privileges beyond what is necessary to execute the tests
5. All code is peer reviewed and tested for vulnerabilities

Mitigation Plan

Software vulnerabilities are any type of weakness in the software that allows for the possibility of a compromise to the confidentiality, integrity, or availability of the system. Vulnerabilities are always going to be present in any software system, so mitigations must be put in place to limit the amount of risk that they cause. The following will discuss the strategies put in place to address any vulnerabilities or software errors that could arise after testing.

Because vulnerabilities are always going to be present in the software, it is essential that continuous monitoring is put in place to identify possible risks early. That is an ongoing process since vulnerabilities can appear anytime in software development. Contoso overall approach will be a continuous cycle that includes risk identification, risk impact assessment, risk analysis, risk tracking, and risk mitigation planning and monitoring. The mitigation plan shall manage and track any identified risks and shall eliminate or reduce the identified risk to an acceptable level. The plan shall be continuously tested and reassessed and revised if needed.



Risks and their relationships will first be defined, and then their probabilities and possible consequences will be assessed; this could include impacts to cost, schedule, performance, etc.

Following the impact assessment, risk prioritization will take place to identify the most critical risk events. Risk tracking then receives input from the risk analysis to provide input to risk mitigation (step 4) and risk impact assessment (step 2).

Risk events that were assessed as high criticality may go into the risk mitigation planning step, while low-risk items may be tracked and monitored. At the end of step 4, the plan and existing risk is reassessed, and new risks are identified as the cycle repeats.

For each of the risks identified in the risk management process, a management approach must be taken in response. There are several management options; these approaches include:

Acceptance – in this case, we have identified the risk and accept that there is a possibility that it could happen, but we will take no action to mitigate it. That would likely be the case for minimal risks that will not have a significant impact on the system. Choosing to adopt this strategy will require program leadership approval.

Avoidance – in this case, we will take steps to avoid the risk entirely. That could mean adjusting the schedule or program requirements to prevent the possibility of a potential risk. That is best used for risks with a potentially significant impact on the project and choosing to take this strategy will require program leadership approval before proceeding.

Transference – this strategy would involve transferring the impact and management of the risk to another party or stakeholder that may be more capable of managing the risk.

Mitigation – this case involves limiting the impact of a risk by putting controls in place to limit its likelihood or impact.

Approvals
There are several layers of approvals in the testing plan, and testing in general. These levels are composed of multiple staff members and include individual Developers, Team Leads, Quality Assurance, and the Change Advisory Board.

A detailed approval breakdown is as follows:

- Developers – Can approve individual changes during peer-review sessions as appropriate
- Team Leads – Can approve significant changes and promote code into different environments except production
- Quality Assurance – Can approve changes for final review before entering production
- Change Advisory Board – Final approval before changes are promoted into production

Sources

Elgabry, O. (2016, September 14). Requirements Engineering - Requirements Validation (Part 4). Retrieved from https://medium.com/omarelgabrys-blog/requirements-engineering-requirements-validation-part-6-29778d7bde24

Firesmith, D. (November 11, 2013). Software Engineering Institute. Retrieved from https://insights.sei.cmu.edu/sei_blog/2013/11/using-v-models-for-testing.html
Guru99. (n.d.). Unit testing tutorial: What is, types, tools, example. Retrieved from https://www.guru99.com/unit-testing-guide.html

Guru99. (n.d.). What is Functional Testing? Types & Examples (Complete Tutorial). Retrieved from https://www.guru99.com/functional-testing.html

Guru99. (n.d.). What is Regression Testing? Definition, Test Cases. Retrieved from www.guru99.com: https://www.guru99.com/regression-testing.html

Mitre. (n.d.). Risk Mitigation Planning, Implementation, and Progress Monitoring. Retrieved from https://www.mitre.org/publications/systems-engineering-guide/acquisition-systems-engineering/risk-management/risk-mitigation-planning-implementation-and-progress-monitoring